SHERLOCK

Security Review For Vesper



Collaborative Audit Prepared For: **Vesper**
Lead Security Expert(s): **montecristo**
oot2k
Date Audited: **October 5 - October 12, 2026**

Introduction

Security Review focusing on Vesper Strategies

When you deposit into a Vesper vault, your funds don't sit idle. The vault spreads them across strategies, and each strategy puts its share to work in one place: a lending market, a Curve pool, a restaking vault and so on. Every strategy reports back what it earned or lost, so your share of the vault tracks what really happened to the money.

Strategies are where funds actually meet the outside world, so that's where we Sherlock looked. This review covers the shared code every strategy is built on, plus Vesper's Cap, StakeDAO/Curve, 3Jane, Morpho and SwapAdapter strategies, a strategy for Vesper2 (our own pegged-token protocol), and how they all work with our vaults.

Vesper wants you to be able to trust where your money goes. This report is part of how they earn that.

Scope

Repository: bloqpriv/vesper-strategies

Audited Commit: 693c0d9638b3388d665163ce107c56e23de9038c

Final Commit: aac8cafad7404a1ef825ca03d6c26aa2942e6c53

Files:

- contracts/interfaces/3jane/IsUSD3.sol
- contracts/interfaces/3jane/IUSD3.sol
- contracts/interfaces/cap/ILender.sol
- contracts/interfaces/cap/IStakerRewards.sol
- contracts/interfaces/cap/ISymbioticDelegator.sol
- contracts/interfaces/cap/ISymbioticVaultFactory.sol
- contracts/interfaces/cap/ISymbioticVault.sol
- contracts/interfaces/curve/IAddressProvider.sol
- contracts/interfaces/curve/ICurve.sol
- contracts/interfaces/curve/ILiquidityGaugeFactory.sol
- contracts/interfaces/curve/ILiquidityGauge.sol
- contracts/interfaces/curve/IMetaRegistry.sol
- contracts/interfaces/curve/ITokenMinter.sol
- contracts/interfaces/morpho/IMorphoVaultV2.sol
- contracts/interfaces/stakedao/IStakeDaoForCurve.sol

- contracts/interfaces/vesper2/IPeggedToken.sol
- contracts/interfaces/vesper2/IVesper2Gateway.sol
- contracts/interfaces/vesper2/IVesper2StakingVault.sol
- contracts/strategies/3jane/SUSD3.sol
- contracts/strategies/3jane/USD3.sol
- contracts/strategies/cap/CapForStETH.sol
- contracts/strategies/cap/Cap.sol
- contracts/strategies/curve/CurveBase.sol
- contracts/strategies/ERC4626Vault.sol
- contracts/strategies/morpho/MorphoVaultV2.sol
- contracts/strategies/stakedao/StakeDao.sol
- contracts/strategies/Strategy.sol
- contracts/strategies/SwapAdapter.sol
- contracts/strategies/vesper2/Vesper2.sol

Final Commit Hash

aac8cafad7404a1ef825ca03d6c26aa2942e6c53

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	5	3

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue M-1: The USD3 strategy calls the removed `minCommitmentTime()` and cannot rebalance against the live USD3 [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-vesper-oct-5th-2026/issues/31

Summary

3Jane removed the USD3 commitment lock (`minCommitmentTime()` and `depositTimestamp()`) in its v1.2 release, and the live USD3 proxy already runs that implementation. Vesper's `USD3` strategy still calls `minCommitmentTime()` unconditionally at the top of `_deposit`, which `ERC4626Vault._rebalance` always executes. Every `rebalance` therefore reverts, so the strategy can never take capital, report profit or loss, or repay excess debt through the normal path. The SUSD3 strategy's lock views fail the same way.

Vulnerability Detail

- `contracts/strategies/3jane/USD3.sol:35`: `if (_usd3.minCommitmentTime() > 0 && pool().excessDebt(address(this)) > 0) return;` This is the first statement of `_deposit` and it runs before any amount check.
- `contracts/strategies/ERC4626Vault.sol:104-106`: `_rebalance` calls `reportEarning` and then unconditionally `_deposit(balance)`.
- `contracts/strategies/3jane/USD3.sol:14-22` and `contracts/strategies/3jane/SUSD3.sol:97-122`: `lockDuration()` and `unlockTime()` call `minCommitmentTime()` / `depositTimestamp()`.
- `contracts/interfaces/3jane/IUSD3.sol:12-16` still declares both functions.

On the 3Jane side, commit `a76f155d` (”refactor(usd3): remove deprecated whitelist, commitment, and migration hooks”, 7 Jul 2026, tag `v1.2`) deleted them. Current `src/usd3/USD3.sol:97` keeps only `mapping(address => uint256) private __deprecated_depositTimestamp;`. Because USD3 is a Yearn tokenized strategy, an unknown selector falls through `BaseStrategyUpgradeable.fallback()` (lines 463-478) into a delegatecall to TokenizedStrategy, which reverts with empty data.

Impact

Both USD3 and sUSD3 integrations will not work in production.

Code Snippet

https://etherscan.io/address/0x056B269Eb1f75477a8666ae8C7fE01b64dD55eCc#readProxyContract

https://github.com/3jane-protocol/moneymarket-contracts/compare/v1.1.4...v1.2.0#diff-47476429eaeb244099600dbc5cfb6777c967d823cf785af6a440398d94732089L752

Tool Used

Manual Review

Recommendation

Rework both strategies to reflect the recent 3Jane update.

Issue M-2: SUSD3 does not check for potential pending losses [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-vesper-oct-5th-2026/issues/32

Summary

During a rebalance, the SUSD3 strategy deposits idle USDC into USD3 (hop 1) and stakes it into sUSD3 (hop 2) with no check for a pending USD3 loss. A pending loss means USD3's live `nav()` has dropped below its reported `totalAssets()` after a credit loss or markdown in MorphoCredit. When 3Jane next reports, USD3's loss waterfall burns USD3 straight out of sUSD3's balance. Vesper's just-deposited capital has been minted at the stale prices and pooled with the existing junior holders, so it absorbs a loss that happened before it arrived.

The strategy already detects this exact condition in the same function: it calls `_hasUnrealizedLoss()` and suppresses profit. It does not use the check to skip the reinvestment. The `USD3` strategy does skip its deposit in this state.

Vulnerability Detail

- `contracts/strategies/3jane/SUSD3.sol:203-204`: `_hasUnrealizedLoss()` only zeroes `_profit`.
- `SUSD3.sol:219-220`: `if (_excessDebt == 0) _deposit(balance)` runs regardless.
- `SUSD3.sol:141-161`: `_deposit` checks only caps, the minimum deposit and the cooldown or lock state, never USD3 or sUSD3 pending loss.
- On the 3Jane side, nothing blocks the deposit:
  - USD3 `availableDepositLimit` (`src/usd3/USD3.sol:523-563`) has no pending-loss gate, unlike `availableWithdrawLimit` (lines 476-478).
  - sUSD3 `availableDepositLimit` (`src/usd3/sUSD3.sol:250-256`) only checks sUSD3's own unreported loss, not USD3's.
  - USD3 `_postReportHook` (`src/usd3/USD3.sol:616-640`) then burns `loss * preSupply / preAssets` shares from sUSD3's USD3 balance, after burning locked profit.

Compare `contracts/strategies/3jane/USD3.sol:31-36`, where the USD3 strategy returns early from `_deposit` when `_hasUnrealizedLoss()` is true.

Impact

The keeper can accidentally deposit strategy funds into USD3 which can have a pending unrealized loss. In this case the new deposit can be fully lost.

Code Snippet

Tool Used

Manual Review

Recommendation

In `SUSD3._rebalance` (or at the top of `_deposit`), skip both hops while `_hasUnrealizedLoss()` is true, mirroring `USD3._deposit`. Leave the capital idle until 3Jane has reported and the mark is current.

Issue M-3: Curve uses the larger accounting mark as executable liquidity [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-vesper-oct-5th-2026/issues/36

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Curve strategies value LP tokens using the maximum of an oracle mark and the current one-coin withdrawal quote. Because they inherit the base `instantLiquidity()` implementation, the larger mark is also advertised as synchronously withdrawable even when Curve can execute only the lower value.

Vulnerability Detail

- `contracts/strategies/curve/CurveBase.sol:216-218` includes `_quoteLpToCollateral` in TVL.
- `contracts/strategies/curve/CurveBase.sol:411-419` returns `max(oracleQuote, curveQuote)`.
- `contracts/strategies/Strategy.sol:151-159` exposes that TVL as instant liquidity.
- `contracts/strategies/curve/CurveBase.sol:532-545` then requires the requested collateral amount, less then only configured slippage, from the executable Curve exit.

Convex and StakeDAO Curve descendants inherit the same behavior.

If 100 LP is worth 100 collateral according to the oracle but Curve imbalance or a depeg reduces `calc_withdraw_one_coin` to 80, the strategy reports 100. With 3% slippage, a request for 100 tries to exit while requiring at least 97; the pool can provide only 80 and reverts.

Impact

Pool imbalance can lead to revert of withdraw, can potentially be exploited by a griefer.

Code Snippet

Tool Used

Manual Review

Recommendation

Keep the accounting and executable-liquidity quote separate.

Discussion

rokso

Acknowledged, no change.

- `instantLiquidity` **is removed.** It only existed for the async vaults, which are retired, so we've removed it from the strategies entirely. The `YieldVault` wraps each strategy's `withdraw` in `try/catch` and moves on to the next one in the queue, so a Curve withdraw that reverts doesn't block the user's withdrawal at that step.
- **Marking at** `max(oracle, calc_withdraw_one_coin)` **is deliberate.** We don't want pool tilt or a short depeg booked as a realized loss. Exits sell only the oracle-sized amount minus slippage. If the pool can't pay that, the withdraw reverts and we wait for the pool to recover. We chose a revert over silently realizing a large loss, after running a lot of tilt and depeg scenarios.

Issue M-4: A keeper stray-cleanup call transfers position value to feeCollector and books a realized loss [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-vesper-oct-5th-2026/issues/37

Summary

`CurveBase` never overrides `isReservedToken()`, so only the Curve LP and the collateral token are protected from `sweep()`. But the strategy treats `_underlyingTokens` (including WETH on ETH-coin pools) and claimed reward tokens as its own value everywhere else: `_getDepositData()` invests their full balances and `_approveToken()` approves them as protocol assets. A single `sweep(USDT)`-style call – most plausible right after `withdrawAllCoins()`, when the whole non-collateral half of the position sits as coins and a stray-cleanup sweep is routine – sends the balance to `feeCollector`, and the next `rebalance()` reports it as a realized loss.

Vulnerability Detail

The additional tokens that need protection from `sweep()` are **the other coins in the configured Curve pool**, besides the strategy's collateral token. Their addresses are stored in `_underlyingTokens`. When `withdrawAllCoins()` removes the strategy's liquidity, it returns these coins to the strategy. They are still user principal, waiting to be swapped back to collateral or reinvested.

For example, a USDC strategy using the eUSD/USDC Curve pool receives both USDC and eUSD on a proportional withdrawal. USDC is already protected as collateral; **eUSD must additionally be protected from sweeping**. Otherwise, `sweep(eUSD)` transfers that part of the withdrawn position to `feeCollector`.

For a Curve pool containing native ETH, the corresponding WETH balance also needs protection if WETH is not already the collateral token, because the strategy wraps ETH received from Curve into WETH. Ordinary reward tokens, such as CRV or CVX, do not need this additional protection unless they are also one of the configured pool coins.

The missing check is in the sweep path: the base reserves only collateral and LP, and StakeDao adds only its RewardVault shares. Non-collateral pool coins therefore pass the guard even when they came from withdrawing the investment. Protect these coins from sweeping while keeping `swapToCollateral()` available to finish the unwind.

Impact

Pool depositors bear the booked loss and the permanent `debtRatio` cut. `feeCollector` defaults to the pool governor, so this is keeper misoperation rather than theft – but it is one ordinary call with position-sized consequences.

Code Snippet

https://github.com/sherlock-audit2/2026-09-vesper-oct-5th-2026/blob/main/vesper-strategies/contracts/strategies/Strategy.sol#L118-L121

https://github.com/sherlock-audit2/2026-09-vesper-oct-5th-2026/blob/main/vesper-strategies/contracts/strategies/stakedao/StakeDao.sol#L59-L62

Tool Used

Manual Review

Recommendation

Override `isReservedToken()` in `CurveBase` to also reserve every `_underlyingTokens` and `_rewardTokens` entry (StakeDao inherits it).

Issue M-5: A keeper drains SwapAdapter principal by repeating oracle-floor round trips [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-vesper-oct-5th-2026/issues/38

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`SwapAdapter.swap()` bounds each principal swap by the oracle quote less `slippage`, but nothing bounds how many swaps a keeper makes or in which direction. A keeper can free target token from the inner strategy, sell it for collateral at the floor, buy target back at the floor, and repeat. When the keeper supplies the liquidity through a whitelisted aggregator, it keeps the slippage on every leg.

Every call passes every check in the adapter, yet the losses compound. With the repository's 50 bps setting, 230 round trips move 85% of a 1,000,000 USDC allocation to the keeper.

The contract documents this exact threat and claims the opposite:

> A keeper cannot route principal away for more than `slippage` bps. (`contracts/strategies/SwapAdapter.sol:282-283`)

The guarantee holds per call. It does not hold across calls.

Vulnerability Detail

Trust Model: The adapter treats the keeper as untrusted where principal routing is concerned. The keeper supplies the router and arbitrary calldata (`SwapAdapter.sol:272-274`, `279-281`). The adapter then constrains the outcome specifically to contain a malicious or compromised keeper:

- a governor router whitelist;
- an exact, immediately revoked approval;
- a snapshot showing the bystander principal token did not fall;
- an oracle floor for principal.

Root Cause:

1. **The floor is per call.** `SwapAdapter.sol:321-330` derives `_floor` from the oracle quote for this call's `amountIn_`. No state records the slippage already realized, so the bound resets on every call.

2. **Both directions are always allowed.** `tokenIn_` and `tokenOut_` may each be collateral or target (`SwapAdapter.sol:299-306`). The buffer check (`SwapAdapter.sol:314-319`) only stops the collateral→target leg from going below the buffer. The target→collateral leg refills it, so the buffer never limits the loop.
3. **The cycle needs no other actor.** `withdrawFromInner` (`SwapAdapter.sol:363-372`) frees any amount of target on demand. `swap` pushes target proceeds straight back to the inner (`SwapAdapter.sol:352`), and the inner's `withdraw` pays from that idle balance. No inner rebalance is needed between rounds.
4. **Realized slippage is never accounted inside `swap`.** Only `_rebalance` (`SwapAdapter.sol:474-500`) measures profit and loss, after the fact. Its only loss cap is the keeper-supplied `maxLoss_`.
5. **Routers can let the caller choose the counterparty.** The natspec names aggregators such as 1inch and LiFi as intended routers. With 1inch's `swap(executor, desc, data)`, the caller chooses the executor and `srcReceiver`. The adapter's `tokenIn` therefore goes to the keeper's contract, which pays back exactly the floor in `tokenOut`. The router only checks that the receiver gained at least `minReturnAmount`. This is the same check the adapter performs.

Example: The pool has allocated `D` collateral to the adapter. Most of it is held as target token in the inner strategy, and the rest stays as the collateral buffer. A whitelisted aggregator router is configured. The keeper repeats:

1. `withdrawFromInner(innerTvl())`: all target moves to the adapter.
2. `swap(target, collateral, amount, floor, router, data)`. The keeper's executor receives the target and pays `quote × (1 − slippage)` collateral.
3. `swap(collateral, target, balance − buffer, floor, router, data)`. The keeper's executor receives the collateral and pays `quote × (1 − slippage)` target. The adapter forwards the target to the inner.

After `N` round trips, roughly `(1 − slippage)^(2N)` of the swapped position remains. The keeper holds the difference.

Configured `slippage`	Round trips to take ~90% of the swapped position
50 bps (repository tests)	230
100 bps	115
1,000 bps (`MAX_CONFIGURABLE_BPS`)	11

Impact

- Pool depositors lose up to the adapter's entire allocation above the collateral buffer, and the keeper receives it.

- The loss is spread across every vault shareholder when it is reported. The pool also cuts the strategy's `debtRatio` on that loss.
- The governor can only react afterwards by removing the keeper or the router.

Code Snippet

Tool Used

Manual Review

Recommendation

Discussion

rokso

Acknowledged. We're fixing this through access control, not contract changes.

Good catch: the slippage floor holds per swap, not across a loop of swaps. An OpenZeppelin `AccessManager` will be SwapAdapter's only keeper: `swap` is restricted to the ops multisig, `rebalance` and `withdrawFromInner` to the keeper bots, and the governor Safe is the admin. Only the multisig can swap principal, so the loop can't be run.

Issue L-1: Curve caps the LP burn at the position but keeps the uncapped min-out, reverting a full exit [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-vesper-oct-5th-2026/issues/33

Summary

When the requested collateral exceeds what the whole LP position can return, `CurveBase._withdrawHere` caps the LP burned at the position size but still derives `minAmountOut` from the original request. The Curve removal then reverts, blocking both loss recognition with payback in `rebalance` and pool withdrawals from a loss-making strategy.

Vulnerability Detail

`contracts/strategies/curve/CurveBase.sol:532-545`:

```
uint256 _lpToBurn = _quoteCollateralToLp(amountToWithdraw_);
_lpToBurn = Math.min(_lpToBurn, totalLp_);
...
uint256 _minAmountOut = (amountToWithdraw_ * (MAX_BPS - $._slippage)) / MAX_BPS;
_withdrawOneCoin($, _lpToBurn, _minAmountOut);
```

After the cap, `_minAmountOut` is not rescaled to the LP actually burned. `BalancerV3Stable._withdrawHere` handles the same case correctly by scaling the expected output to the BPT actually burned.

Impact

The rebalance cannot report a loss or repay any debt, and a pool `withdraw` for the full debt of the strategy reverts in the same way.

Workarounds exist (restore the debt ratio, report the loss first, or raise slippage).

Code Snippet

Tool Used

Manual Review

Recommendation

When `_lpToBurn` is capped, compute `minAmountOut` from the value of the LP actually burned, e.g. `calc_withdraw_one_coin(_lpToBurn) * (MAX_BPS - slippage) / MAX_BPS`, floored by the oracle value of `_lpToBurn`.

Issue L-2: SwapAdapter exposes swapped collateral before booking the realized loss [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-vesper-oct-5th-2026/issues/34

Summary

The standard ERC-4626 and Curve strategies unwind assets and report accounting inside one `rebalance` transaction. `SwapAdapter` instead splits an unwind into independent keeper calls to `withdrawFromInner`, `swap`, and `rebalance`.

`withdrawFromInner` can realize an exit fee or shortfall, and `swap` can legitimately execute below the oracle price. Neither call updates the outer vault's debt. After `swap` returns, however, all received collateral is immediately transferable through `Strategy.withdraw()` and is also shown by `instantLiquidity()`. A shareholder can therefore redeem at the stale pre-loss share price before the later `rebalance` assigns the loss to whoever remains.

Vulnerability Detail

- `contracts/strategies/SwapAdapter.sol:363-372` accepts the amount actually returned by the inner strategy but does not reconcile any reduction in the combined position value.
- `contracts/strategies/SwapAdapter.sol:324-354` permits a principal swap to execute at the oracle quote minus `slippage`, verifies the resulting balances, and returns without reporting the realized value change.
- `contracts/strategies/SwapAdapter.sol:174-176` immediately counts all output collateral as withdrawable liquidity.
- `contracts/strategies/SwapAdapter.sol:474-500` is the only adapter path that updates outer profit/loss, and it is a separate keeper transaction.
- `contracts/strategies/Strategy.sol:287-300` lets the pool take the newly available collateral without first invoking adapter accounting.

By comparison, `ERC4626Vault._rebalance` and `CurveBase._rebalance` free the collateral required by their report and call `pool.reportEarning` before the rebalance transaction completes.

Impact

A user withdrawing between swap or withdrawFromInner and rebalance will not bear any potential loss. The bad debt will be socialized among the later withdrawing users leading to incorrect loss accounting.

Code Snippet

Tool Used

Manual Review

Recommendation

Update potential losses similar to rebalance.

Issue L-3: Out of gas on Symbiotic reward claims can make rewards unclaimable [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-vesper-oct-5th-2026/issues/35

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`Cap._claimRewards()` always asks Symbiotic to process every unclaimed reward distribution in one transaction. Because the cost of the external claim grows with the number of distributions, a sufficiently large backlog can exceed the block gas limit. Both of the strategy's reward-claiming paths use this helper, so keepers cannot recover by claiming a smaller batch.

Vulnerability Detail

The strategy calculates the difference between `rewardsLength` and `lastUnclaimedReward` and encodes that entire value as Symbiotic's `maxRewards` argument. It also supplies an empty `activeSharesOfHints` array. Symbiotic then iterates over the requested distributions and resolves the strategy's historical active shares for each one, making claim gas consumption grow with the unclaimed history.

```
uint256 _unclaimed = stakerRewards_.rewardsLength(token_, network_) -
    stakerRewards_.lastUnclaimedReward(address(this), token_, network_);

if (_unclaimed == 0) return 0;

// Prepare claim data: (network, maxRewards, activeSharesOfHints)
bytes memory _data = abi.encode(network_, _unclaimed, new bytes[](0));

stakerRewards_.claimRewards(address(this), token_, _data);
```

Impact

In case rewards are not claimed, future reward claims revert.

Code Snippet

Tool Used

Manual Review

Recommendation

Process rewards in batches

Discussion

rokso

Acknowledged, won't fix.

The finding is right in theory: a claim processes every unclaimed distribution in one call, so a big enough backlog could exceed the gas limit. In practice it isn't reachable:

- Each distribution adds roughly 13k gas. On the live `Cap_wstETH_vastETH` strategy, claims cost about 410–430k gas for 1 distribution, ~460k for 4 and ~510k for 8 (including the reward swap).
- At the 16.7M per-transaction cap (EIP-7825), one claim can clear about 1,250 distributions.
- The network has published 40 distributions so far, about 5 a month, and our bot claims regularly. Reaching the limit would take about 20 years with no claims at all.

If it ever happened, only reward claiming is affected. rebalance doesn't claim, so principal and accounting keep working.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.